Universidade Positivo

DESENVOLVIMENTO WEB BÁSICO

Aula: DWB.09 – Javascript: Funções. Estrutura Condicional. Estrutura de Repetição. Array.

Professor: Fabricio Olivo
fabricio@up.edu.br

JAVASCRIPT

OPERADORES RELACIONAIS

Operador	Nome	Exemplo	Resultado
==	Igual	a == b	Verdadeiro se a for igual a b
!=	Diferente	a != b	Verdadeiro se a não for igual a b
===	Idêntico (igualdade estrita)	a === b	Verdadeiro se a for igual a b e for do mesmo tipo
!==	Não idêntico (diferença estrita)	a !== b	Verdadeiro se a não for igual a b, ou eles não são do mesmo tipo
<	Menor que	a < b	Verdadeiro se a for menor que b
>	Maior que	a > b	Verdadeiro se a for maior que b
<=	Menor ou igual	a <= b	Verdadeiro se a for menor ou igual a b.
>=	Maior ou igual	a >= b	Verdadeiro se a for maior ou igual a b.

JAVASCRIPT

OPERADORES RELACIONAIS

```
let igualdade = 2 == '2'; //
console.log('igualdade: ', igualdade); // true

let diferenca = 2 != '2'; //
console.log('diferenca: ', diferenca); // false

let igualdadeEstrita = 2 === '2';
console.log('igualdadeEstrita:', igualdadeEstrita); // false

let diferencaEstrita = 2 !== '2';
console.log('diferencaEstrita:', diferencaEstrita); // true
```

JAVASCRIPT

OPERADORES LÓGICOS

&& (E)
(retorna verdade somente se todas as condições são verdadeiras)

|| (OU)
(retorna verdade se pelo menos uma das condições for verdadeira)

! (Negação)
(inverte o valor da expressão)

```
let condicaoE = true && true; // e
console.log('true && true:', condicaoE);

condicaoE = true && false; // e
console.log('true && false:', condicaoE);

let condicaoOU = false || true; // ou
console.log('false || true:', condicaoOU);

condicaoOU = false || false; // ou
console.log('false || false:', condicaoOU);

let negacao = !false; // negação
console.log('!false:', negacao);
```

JAVASCRIPT

ESTRUTURAS CONDICIONAIS

- Estruturas condicionais ajudam a **controlar o fluxo** de execução do código, permitindo que diferentes blocos de instruções sejam executados **dependendo** do resultado de uma **condição**.
- As estruturas condicionais tem a **capacidade** de tornar o programa **dinâmico** e **flexível**.
- **if**: permite a execução de um bloco de código **se** uma **condição** específica for **verdadeira**.
- **else if**: permite a verificação de outras condições após um if inicial, possibilitando a execução de diferentes blocos de código para cada condição específica.
- **else**: permite a execução de um bloco de código alternativo **se** a **condição** do if for **falsa**.
- **switch-case**: permite a execução de um bloco de código específico com base no valor de uma variável.

JAVASCRIPT

EXEMPLO DE IF ELSE IF ELSE

- Escreva um código javascript que **leia** o valor de uma compra, **calcule** o desconto e **apresente** na tela o valor da compra, o desconto e o valor da compra com desconto.
- Critérios para desconto:
  - desconto de 10% para valores até 100 reais,
  - desconto de 15% para valores entre 100 e 500 reais, e
  - desconto de 20% para valores acima de 500 reais.

```
let valorCompra = prompt("Digite o valor da compra: ");

valorCompra = Number(valorCompra);

let desconto = 0;

if (valorCompra <= 100) {
    desconto = valorCompra * 0.10;
}
else if (valorCompra > 100 && valorCompra <= 500) {
    desconto = valorCompra * 0.15;
}
else {
    desconto = valorCompra * 0.20;
}

let valorFinal = valorCompra - desconto;

console.log('Valor da compra: R$ ' + valorCompra);
console.log('Desconto aplicado: R$ ' + desconto);
console.log('Valor final da compra: R$ ' + valorFinal);
```

JAVASCRIPT

EXEMPLO DE SWITCH-CASE

- Escreva um código javascript que **leia** o dia da semana abreviado e **apresente** na tela uma mensagem como indicado:
  - para “**segunda**” e “**terça**” apresente “Mais café, por favor!”,
  - para “**quarta**” e “**quinta**” apresente “A semana está no meio, quase lá!”,
  - para “**sexta**” apresente “Finalmente sexta-feira!”, e
  - para “**sábado**” e “**domingo**” apresente “Fim de semana, que beleza!”.

```
let diaDaSemana = prompt("Informe o dia da semana: ");

switch (diaDaSemana) {
    case "segunda":
    case "terça":
        console.log("Mais café, por favor!");
        break;
    case "quarta":
    case "quinta":
        console.log("A semana está no meio, quase lá!");
        break;
    case "sexta":
        console.log("Finalmente sexta-feira!");
        break;
    case "sábado":
    case "domingo":
        console.log("Fim de semana, que beleza!");
        break;
    default: console.log("Informe: segunda, terça,
quarta, quinta, sexta, sábado ou domingo!");
}
```

ESTRUTURAS DE REPETIÇÃO: EXEMPLO DE LAÇO FOR

- O laço **for** possui três partes principais que ficam entre parênteses e separadas por vírgulas: a **inicialização**, a **condição** de **continuidade** e a **expressão** de **incremento**.
- Exemplo:

```
let precoCarros = [18050, 12000, 14200, 20000, 12000, 8250, 9500];

let soma = 0;
for (let i = 0; i < precoCarros.length; i++) {
        soma = soma + precoCarros[i];
}
console.log('Estoque total: R$', soma); // 94000
```

JAVASCRIPT

ESTRUTURAS DE REPETIÇÃO: EXEMPLO DE LAÇO WHILE

- O laço **while** é uma estrutura de repetição que permite executar um bloco de código enquanto uma condição for verdadeira.
- O **while** é útil quando o número de iterações não é conhecido e depende da entrada do usuário.
- Exemplo:

```
<html>
  <head>
    <title>do-While em JavaScript</title>
  </head>
  <body>
    <button onclick="logar()">Entrar</button>
    <script src="script.js"></script>
  </body>
</html>
```

```
function logar(){
  let senha = prompt("Digite sua senha: ");
  while(senha !='2026js'){
    if(senha != '2026js')
      alert("Senha invalida, tente novamente!");
   senha = prompt("Digite sua senha: ");
  }
  alert("Entrando no sistema...");
}
```

JAVASCRIPT

ESTRUTURAS DE REPETIÇÃO: EXEMPLO DE LAÇO DO-WHILE

- O laço **do-while** permite executar o bloco de código antes de verificar a condição.
- São muito usados para exibir menu, pedir dados, senhas...
- Exemplo:

```
<html>
  <head>
    <title>do-While em JavaScript</title>
  </head>
  <body>
    <button onclick="logar()">Entrar</button>
    <script src="script.js"></script>
  </body>
</html>
```

```
function logar(){
  let senha = '';
  do{
    senha = prompt("Digite sua senha: ");
    if(senha != '2026js')
      alert("Senha invalida, tente novamente!");
  }while(senha !='2026js');
  alert("Entrando no sistema...");
}
```

JAVASCRIPT

ESTRUTURA DE DADOS (ARRAY)

- Índice do ***array*** sempre começa em 0.
- Declaração é feita entre colchetes "**[ ]**" e virgula "," para separar os elementos.
  - `let numeros = [10, 20, 30, 40, 50, 60, 70];`
  - `console.log(numeros.length);`
  - `console.log(numeros);`
  - `console.log(numeros[2]);`
  - `console.log(numeros.indexOf(40)); //procura o valor no array e retorna seu índice caso encontrado ou retorna -1 caso não encontrado`
  - `console.log(numeros.includes(60)); //procura pelo valor no array e retorna 'true' caso encontrado ou 'false' caso não encontrado`

JAVASCRIPT

ESTRUTURA DE DADOS (ARRAY)

- ***Arrays*** tem **métodos** usados para lidar com seus dados:

```
let pessoas = ["João", "Maria", "José"];
pessoas.unshift("Bruno", "Rafael"); //add dados no início do array
pessoas.push("Paulo", "Gabriel"); //add dados no fim do array
pessoas.shift(); //exclui e retorna o primeiro dado do array
pessoas.pop(); //exclui e retorna o último dado do array
let pos = pessoas.indexOf("João"); //recupera o índice do dado no array (posição)
pessoas.splice(pos , 2, "George") // O método splice() altera o conteúdo de uma lista, adicionando novos elementos enquanto remove elementos antigos. O primeiro parâmetro (pos) é a posição a partir da qual pretendemos remover os elementos, o segundo parâmetro (2) é a quantidade de elementos que pretendemos remover.
```

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Array

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/slice

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/toSorted

JAVASCRIPT

ESTRUTURA DE DADOS (ARRAY)

- Exemplo de Arrays de **Objetos**:

```
// Array com nomes e notas dos alunos
let alunos = [  { nome: "Alice",   nota: 92 },
                { nome: "Bento",   nota: 38 },
                { nome: "Carol",   nota: 88 },
                { nome: "Marcos",  nota: 65 },
                { nome: "Elaine",  nota: 72 }  ];
```

- `console.log(alunos);`
- `console.log(alunos[2]);`
- `console.log(alunos[2].nome);`
- `console.log(alunos[2].nota);`
- `console.log(alunos.length);`

ATIVIDADE 1

- Escreva um código em javascript que **leia** três notas de um aluno, **calcule** a media das notas e **informe** a situação do aluno, conforme abaixo:
  - **APROVADO** para média maior ou igual a 6;
  - **REPROVADO** para média menor que 4; ou
  - **RECUPERAÇÃO** para outras médias.

ATIVIDADE 2

Escreva um código em javascript que **inicialize** uma **estrutura de dados** com o nome e nota de 5 alunos.
Crie botões para executar cada uma das seguintes funções:

- **função** “listaAlunos( )” para **exibir** na página os dados de todos os alunos;
- **função** “maiorNota( )” para **exibir** na página os dados do aluno com MAIOR nota;
- **função** “menorNota( )” para **exibir** na página os dados do aluno com MENOR nota;

```
// use o método forEach() para acessar os dados do array
alunos.forEach(function(item, indice, array){
    console.log("O aluno " + item.nome + ` teve nota ${item.nota}`);
});
```

ATIVIDADE 3

1. Escreva um código em javascript para **inicializar** uma **estrutura de dados** com o nome, idade, altura, e esporte de 7 atletas.
   Crie botões para executar cada uma das seguintes funções:
   - **função** para **calcular** e **exibir** na página os dados de todos os atletas;
   - **função** para **calcular** e **exibir** na página os dados do atleta mais alto;
   - **função** para **calcular** e **exibir** em tela os dados do atleta mais velho.

ATIVIDADES

1. Gerar um arquivo compactado (ZIP) a partir dos arquivos produzidos na aula.
2. Atribuir seu nome e data e **enviar** o arquivo compactado para o ***link*** de entrega no **BB**.

TUTORIAIS E GUIAS

- Confira essas matérias sobre os fundamentos da WEB:
  - Um guia para iniciantes na área de desenvolvimento web.
  - W3C: Consórcio que regulamenta os padrões para web.

- Por aqui você começará a entender como são estruturadas as interfaces das páginas dos sites na Internet:
  - https://developer.mozilla.org/pt-BR/docs/Learn/HTML/Introduction_to_HTML/Getting_started
  - https://www.w3schools.com/
  - Projeto Khan Academy para Programação

- Faça seus testes no desenvolvimento WEB usando ambientes online preparados pra isso:
  - jsbin.com – Testes de Front-End (html, css e js);

BIBLIOGRAFIA BÁSICA

- FLATSCHART, F. **HTML 5: embarque imediato**. Rio de Janeiro: Brasport, 2011. E-book.
- SANDERS, B. **Smashing HTML5: técnicas para a nova geração da web**. Porto Alegre: Bookman, 2012. E-book.
- TERUEL, E. C. **HTML5: guia prático**. 2. ed. São Paulo: Érica, 2014. E-book.

BIBLIOGRAFIA COMPLEMENTAR

- BERTAGNOLLI, S.; MILETTO, E. M. (org.). **Desenvolvimento de software II: introdução ao desenvolvimento web com HTML, CSS, JavaScript e PHP**. Porto Alegre: Bookman, 2014. E-book.
- HAROLD, E. R. **Refatorando HTML: como melhorar o projeto de aplicações web existentes**. Porto Alegre: Bookman, 2010. E-book.
- HORSTMANN, C. **Conceitos de computação com Java**. 5. ed. Porto Alegre: Bookman, 2009. E-book.
- KALBACH, J. **Design de navegação web: otimizando a experiência do usuário**. Porto Alegre: Bookman, 2009. E-book.
- MACHADO, R. P.; FRANCO, M. H. I.; BERTAGNOLLI, S. C. **Desenvolvimento de software III: programação de sistemas web orientada a objetos em Java**. Porto Alegre: Bookman, 2016. E-book.